

Student Semester Average Reflection Log

Understanding the Problem

I approached understanding the challenge by recognizing that the program needed to create a GUI where users can input student information, calculate an average, and manage file data (save, view, and delete).

At first, I found it confusing how multiple buttons performed different actions. I resolved this by understanding how each `ActionListener` is connected to a specific task.

Code reference:

```
savebtn.addActionListener(e -> {  
  
showbtn.addActionListener(e -> {  
  
clearbtn.addActionListener(e -> {
```

Planning the Solution

Before coding, I broke the program into parts: building the GUI layout, handling input, calculating averages, saving to a file, displaying file contents, and clearing the file.

I chose Swing components for the interface and file classes like `BufferedReader` and `BufferedWriter` for file handling.

Code reference:

```
50      frame = new JFrame("Student Semester Average"); //title added  
184      BufferedWriter writeFile = new BufferedWriter(new FileWriter(dataFile, true));
```

Implementation

I created the GUI first by adding labels, text fields, and buttons. Then I implemented the logic for calculating averages and handling file operations.

I tested the program by entering values, checking the calculated average, and verifying that the data was saved and displayed correctly.

Code reference:

```
179     double avgGrade = (Double.parseDouble(grade1) + Double.parseDouble(grade2) +  
180         Double.parseDouble(grade3) + Double.parseDouble(grade4)) / 4;  
181  
182     avg.setText(decimalFormatter.format(avgGrade) + "%");
```

Overcoming Challenges

The most difficult part was validating user input and preventing errors when non-numeric values were entered. Managing file operations like reading and clearing the file was also challenging.

I handled this by using try-catch blocks and checking for empty fields before processing.

Code reference:

```
172⊖    if (studentname.isEmpty() || gradelevel.isEmpty() || semesternum.isEmpty() ||  
173        grade1.isEmpty() || grade2.isEmpty() || grade3.isEmpty() || grade4.isEmpty()) {  
174        JOptionPane.showMessageDialog(null, "Fill all fields first!");  
175        return; //stops execution  
176    }  
  
191⊖    } catch (Exception err) {  
192        JOptionPane.showMessageDialog(null, "Invalid input!");  
193    }
```

Learning

I learned how to build a more advanced GUI with multiple buttons and features. I also improved my understanding of file handling and input validation in Java.

Code reference:

```
199⊖    try (BufferedReader readFile = new BufferedReader(new FileReader(dataFile))) {  
200        String line;  
201⊖        while ((line = readFile.readLine()) != null) {  
202            dis.append(line + "\n");  
203        }  
  
210⊖    try (FileOutputStream fos = new FileOutputStream(dataFile)) {  
211        dis.setText("File cleared.");
```